Journal of Computer Languages 84 (2025) 101329

Contents lists available at ScienceDirect

Journal of Computer Languages

journal homepage: www.elsevier.com/locate/cola

Reviews

The role of data transformation in modern analytics: A comprehensive survey

Sanae Borrohou *, Rachida Fissoune, Hassan Badir

IDS Team, Abdelmalek Essaadi University, Tangier, Morocco

ARTICLE INFO

Keywords:
Data transformation
Data processing
Data cleaning
Data quality
Transformation process
Machine learning

ABSTRACT

Data transformation is a fundamental step in modern data analytics, enabling the conversion of raw data into structured, high-quality formats suitable for analysis. This process plays a crucial role in data cleaning, integration, and preprocessing, ensuring consistency across diverse data sources while addressing challenges such as missing values, inconsistencies, and redundancy. By applying techniques such as scaling, normalization, encoding, feature extraction, and aggregation, data transformation enhances the accuracy and efficiency of analytical and machine learning models. This study provides a comprehensive survey of data transformation techniques, categorizing them into key types: data cleaning and preprocessing, normalization and standardization, feature engineering, encoding categorical data, data augmentation, discretization and data aggregation. We analyze their impact on data quality and explore their interdependencies, presenting a structured framework that connects these transformations within the broader data preprocessing workflow. Additionally, we highlight the challenges of implementing transformation methods in large-scale, heterogeneous datasets, including data integration complexities, security concerns, and resource constraints. By synthesizing recent advancements in the field, this research offers a structured reference for data scientists and researchers, guiding them in selecting appropriate transformation strategies based on their specific analytical needs. Future work will focus on developing a complete data cleaning workflow that integrates transformation techniques for large-scale applications, emphasizing automation and scalability in modern analytics.

1. Introduction

Today's data-prolific world necessitates the need for sophisticated tools and frameworks that can quickly turn data into information. Rapid insights, thus, mean quicker decisions and a competitive edge. But before insights can be unveiled, the raw data must be conditioned and transformed into something that makes sense [1]. Data transformation, together with preparation, cleansing, imputation, quality enhancement, and various other techniques, aims to satisfy this overarching goal [2].

Data transformation is of vital importance in the world of data science, directly affecting both predictive and descriptive analytics [3]. Analytics, though, are only as good as the data on which they are built; therefore, clear insights depend as much on the transformation of data as they do on sophisticated modeling techniques [4]. As such, the foremost requirement for any data product is data that is fit for consumption. Improved techniques in extraction and storage have made data generation an easy task, hence our world of big data, but this data is unstructured and often bounded. Data quality, though improving, remains a challenge limiting any analytics solution, meaning a high degree of data conditioning is necessary for any modeling techniques [5]. The data transformation process thus undertaken ultimately has a bearing on the outcome of the analysis.

Our paper is structured as follows. Section 2 explores the foundations of data transformation, discussing its definition, importance, and classification into key transformation types. Section 3 delves into the techniques and tools used in data transformation, covering essential processes such as data cleaning and preprocessing, normalization and standardization, feature engineering, encoding categorical data, data augmentation, discretization and data aggregation. Section 4 examines applications and challenges in modern data transformation, addressing critical issues such as data integration, quality assurance, scalability, security, and resource management. Section 5 discusses the role of domain-specific languages and general-purpose languages in data transformation, emphasizing how these languages facilitate various transformation processes and improve efficiency. Section 6 presents a synthesis and discussion of the various transformation techniques, highlighting their interdependencies and impact on analytical workflows. Finally, the conclusion outlines future perspectives, with a particular focus on developing a complete data-cleaning workflow that integrates transformation techniques for large-scale datasets.

* Corresponding author.
E-mail addresses: sanae.borrohou@etu.uae.ac.ma (S. Borrohou), rfissoune@uae.ac.ma (R. Fissoune), badir.hassan@uae.ac.ma (H. Badir).

https://doi.org/10.1016/j.cola.2025.101329
Received 14 February 2025; Received in revised form 4 May 2025; Accepted 4 May 2025
Available online 20 May 2025
2590-1184/© 2025 Published by Elsevier Ltd.

2. Foundations of data transformation

Data transformation is the art and science of turning raw data into processed data. It is a fundamental part of most, if not all, analytical work. Without properly transforming data, it is impossible to understand their meanings, relationships, and insights [6]. If one cannot understand the data, then one cannot be sure that any decisions based on that data — such as where to invest money, which consumer segment to target, or how to optimize an operational process — will be any good. Transformation is also critical because data often need to be used for different purposes than the original owner or collector of the data, carefully selected and stored in a manner that ensures their security and recoverability. For different purposes, different data shapes and quality will be necessary [7]. Hence, the transformation process is the means by which the original data can become useful to a different user.

Data transformation is a comprehensive process involving various tasks that can be categorized by purpose, such as normalization and data cleansing; by input and output orientation, like attribute-oriented versus data-oriented transformation; and by methodology level, including unsupervised and supervised approaches. The term "data transformation" is often used interchangeably with terms like pre-processing, cleaning, and wrangling, depending on the techniques involved [8]. This section will explore the basic principles of data transformation, laying the groundwork for understanding more advanced techniques later on.

2.1. Definition and importance

Data transformation identifies a process wherein data is manipulated or modified to enable data quality improvements and enhance decision-making processes. It holds significance as the quality and completeness of datasets directly impact the effectiveness of modern analytics processes. Given their correlation, data transformation is central to erasing any gaps between raw data and comprehensive yet easy-to-comprehend insights [9]. As a result, data transformation should require more than just technical know-how: a step solely handled by professionals possessing high-level business acumen. Throughout this process, it is critical to verify whether data transformation procedures are undertaken with complete adherence to data integrity policies or require additional terminations. Contemporary organizations, irrespective of their services and scale, are embracing a context where data reigns supreme. Of the various requirements that process raw data into coherent information, 'data transformation' assumes noteworthy importance. By knowing what data transformation is and how data transformation works, the first sub-section seeks to give a comprehensive framework [10]. Understanding the role of data transformation is significant because it highlights its purpose, relevance, and potential impact on data analytics. By elaborating on this topic, the section clarifies the scope of data analytics and identifies the function that connects raw data to business intelligence. Ultimately, a cleaner dataset leads to improved data quality and more effective analytics [1,11].

2.2. Types of data transformation

Data transformation approaches have significantly evolved over time to accommodate the diverse needs of both technical and non-technical users. Traditionally, scripting-based tools such as SAS, R, and Python have been the go-to solutions for performing complex transformations. These languages provide extensive flexibility, enabling users to write custom scripts for detailed and precise manipulation of data. However, they also require advanced programming skills and a deep understanding of the data, making them less accessible to users without technical expertise [12]. This reliance on scripting can become a bottleneck, especially in organizations where data analysts or business users need to make rapid decisions based on the data.

To address this gap and democratize data access, low-code and no-code platforms have transformed data transformation by making complex processes more accessible through user-friendly interfaces. Tools like Wrangler allow users to interactively manipulate data with instant previews, streamlining the transformation process [13]. Visual tools such as SOMNUS help improve understanding and debugging of data workflows by illustrating them intuitively [12]. Additionally, guided query builders like Dexteris support ad-hoc transformations through interactive dialogues, while Transform-Data-by-Example (TDE) simplifies data preparation by enabling users to search for existing transformation logic [14]. Platforms like Talend, Alteryx, and Power Query further simplify data integration and transformation, offering visual interfaces for tasks like ETL.

To summarize, modern data transformation approaches cater to both technical and non-technical users, enhancing efficiency through a mix of scripting flexibility, visual tools, and guided query builders. These advancements streamline workflows and help democratize access to valuable data insights across various domains.

3. Techniques and tools for data transformation

There are several data transformation techniques and tools that can help structure and clean data prior to analysis. Data transformation tools and frameworks can be grouped based on their functionalities. In general, data transformation tools support major functions such as data integration, transformation and cleaning, filtering, transformation process design and tracking, extraction of specific data, visualization of complex data and its analysis, and data transformation process scheduling. Depending on the toolset used and the goal of the analysis, researchers can use different data transformation schemes. The effectiveness of data transformation is primarily determined by the size and type of data, integration tool options and required features, as well as user proficiency and objectives. In general, many data-intensive companies consider user-oriented toolsets and algorithms tailored to the features and requirements of the organization to support efficient data transformation. Below are some of the most commonly used techniques and tools in data transformation.

Data transformation tools vary in their strengths and are used across different scenarios depending on the specific needs of organizations. Apache Nifi excels in real-time data routing and stream processing, ideal for organizations needing flexible data flows but requires tuning for larger setups. Talend is a comprehensive ETL tool known for its wide range of connectors and strong data integration capabilities, although it can be resource-heavy for big data. Apache Spark is a top choice for distributed computing and big data analytics due to its speed, though it demands more expertise and infrastructure. For SQL-based transformations within data warehouses, dbt is a simple and powerful tool, although limited to SQL environments. Informatica PowerCenter is well-suited for enterprise-scale ETL and complex transformations, but it is expensive and requires skilled users. AWS Glue and Microsoft Azure Data Factory are cloud-native ETL services with strong integrations into their respective ecosystems, with AWS Glue focusing on serverless, big data processing, and Azure Data Factory excelling in hybrid cloud integration.

Fivetran is highly automated for data extraction and loading, making it ideal for quick deployment, but it offers limited transformation features. Matillion focuses on cloud-native ETL, supporting platforms like Snowflake and Redshift with pre-built connectors, while Apache Airflow is highly customizable for orchestrating complex workflows, though it requires more technical setup. Each tool fits different transformation needs, ranging from real-time processing, cloud-native ETL, and large-scale distributed analytics to hybrid and SQL-based transformations (Table 1).

There are a number of techniques to transform a set of data. Before modeling any data, it is necessary to preprocess the data. Data preprocessing includes a wide range of activities and techniques. Data

Table 1
Comparison of data transformation tools.

Tool	Description	Strengths	Weaknesses
Apache NiFi	Real-time data integration and flow management tool for routing and transforming	Real-time data routing, stream processing, flexible data flows	Requires tuning for large setups
Talend	Comprehensive ETL platform for data integration	Comprehensive ETL, wide range of connectors, strong integration	Resource-heavy for big data
Apache Spark	Distributed computing engine optimized for big data analytics	Speed, distributed computing, big data analytics	Demands expertise and infrastructure
dbt	Data transformation tool designed for SQL-based workflows within data warehouses	Simple and powerful for SQL-based transformations	Limited to SQL environments
Informatica Power Center	Enterprise-grade ETL platform for handling complex transformations	Enterprise-scale ETL, handles complex transformations	Expensive, requires skilled users
AWS Glue	Serverless ETL service integrated with AWS ecosystem	Serverless, integrates with AWS, big data processing	Limited to AWS infrastructure
Azure Data Factory	Cloud-based ETL service for hybrid data integration	Hybrid cloud integration, strong within Azure ecosystem	Tied to Azure ecosystem, learning curve
Fivetran	Automated tool for extracting and loading data with minimal configuration	Highly automated for extraction and loading, quick deployment	Limited transformation features
Apache Airflow	Workflow orchestration tool for managing complex data pipelines	Highly customizable, orchestrates complex workflows	Requires technical setup, steep learning curve

cleaning is recognized as the most important technique, which has an influential effect on the quality of results [15]. After cleaning the data, there are two common techniques to transform data, including normalization and standardization. Normalizing data refers to squishing all data from all fields into a range from 0 to 1. This makes it easier to compare different fields based on their values or contributes to optimization [16]. Standardizing is transforming the data to fit into a normal distribution [17]. Feature engineering can help improve machine learning models and analytics results, decrease computational and memory complexity of the learning process, and simplify the modeling learning steps [18].

Data transformation refers to the process of converting data from one format or structure into another format or structure. This process often includes, but is not limited to, the extraction of a particular subset of data, converting data to another data type, and modifying the data. Different data transformation techniques are available, and each aims to transform data to have different formats and to serve various purposes. The choice of technique will depend on the purpose, file formats, and file structures. It is also influenced by the tools that are intended to serve the transformation activities in an organization [19]. In other words, the domain of traditional data transformation is restricted to structured data. Companies have been limited in their choice of data transformation tools under the restriction of data types. There are many data transformation tools in the market for manipulating and transforming datasets. In general, it is difficult to find comprehensive software covering almost all requirements of data transformation. Each tool offers different functions based on specified objectives. A company should select the tool that most accurately meets its requirements for the transformation tasks.

3.1. Data cleaning and preprocessing

Over the past years, organizations have collected data from various subsystems either manually or automatically. This data could be categorized into two categories: clean data and dirty data. The dirty data consists of information with poor quality. This is a frequent case, as data inconsistency arises owing to either human error or system malfunction. The raw data often encounters quality issues such as noise, incompleteness, redundancy, inconsistency, and inaccuracy, which contribute to less confident or wrong decisions [20]. Thus, in data cleaning, we need to address these kinds of quality issues. Data cleaning is important because the quality of analytical results and models is directly affected by the quality of the data used. Poor quality data can result in poor quality models and results. In practical scenarios, common data quality issues that can be found include outlier values, missing values, and duplicate data.

1. **Outlier Detection** approaches can be categorized into various groups depending on their underlying principles and techniques (Table 2).
   Statistical approaches involve computations such as mean, median, and standard deviation to pinpoint data points that exhibit significant deviations from the expected pattern. Recent algorithms like Extreme Studentized Deviate (ESD) and Grubbs' test are commonly used in this category [32]. Distance-based techniques evaluate the distances between data points to flag those that are the farthest from the cluster's central tendency. Well-known methods such as K-Nearest Neighbors (KNN) outlier detection and Mahalanobis Distance are widely applied for this purpose [33]. Density-based methods concentrate on regions with lower data density, marking data points in sparse areas as potential outliers. The popular LOF, COF, RDF, LOCI, and INFLO are examples of algorithms from this category [34]. Clustering-based methods segment the data into clusters and designate data points outside these clusters as outliers, with algorithms like k-Means and OPTICS being frequently used [32]. Machine learning-based methods utilize algorithms like Support Vector Machines (SVM), Random Forests, and autoencoders to discern patterns in data, identifying deviations as outliers [34]. Additionally, ensemble methods combine multiple detection techniques, like Isolation Forest and Random Cut Forest, to enhance overall accuracy and robustness [32]. A comprehension of these classifications enables researchers and practitioners to select suitable outlier.
2. **Missing Data** patterns describe which values are missing and observed in a data set (Table 3). While no universally defined list exists in the literature, several studies highlight three commonly discussed patterns: univariate, monotone, and non-monotone [35]. Univariate missing data impacts only one variable, making it relatively easy to manage. Monotone missing data follows a sequential order, where missing values in one variable lead to others, facilitating easier handling. In contrast, non-monotone missing data presents no clear pattern, complicating the imputation process and often requiring more advanced techniques to address.

Table 2
Comparative table of different outlier detection approaches.

Approach	Algorithms	Strengths	Weaknesses
Statistical	– Z-Score/Standard Deviation – Grubbs' Test – Dixon's Q Test – IQR (Interquartile Range)	– Providing a clear interpretation of outliers within specific data distributions. – Assigning scores or confidence intervals to data points, enhancing decision-making. – Operating in an unsupervised manner, eliminating the need for labeled data.	– Assume a specific data distribution, which is frequently inaccurate. – This assumption is especially problematic in high-dimensional datasets. – Creating appropriate hypothesis tests for intricate distributions is challenging.
Distance-based	– KNN – Mahalanobis Distance – Minkowski Distance	– Operating without supervision, making no assumptions about data distribution – Adapting these methods to different types of data by choosing an appropriate distance metric.	– Normal instances with few neighbors or anomalies with many neighbors can be mislabeled. – Calculating distances between all data points makes the testing phase computationally expensive. – The performance depends on choosing the right distance metric and tuning the parameters correctly.
Density-based	– LOF – COF – RDF – LOCI – INFLO	– Compared to distance-based methods, density-based outlier detection approaches are generally more effective. – They are capable of detecting anomalies in complex data structures.	– Improvement in efficiency is accompanied by increased complexity and higher computational costs. – The performance of these methods is sensitive to parameters such as the number of neighbors considered.
Clustering-based Approaches	**Partitioning Clustering:** K-Means, Fuzzy C-Means, CLARA, PAM, CLARANS, Affinity Propagation,etc **Density-based clustering:** DBSCAN, DENCLUE, Mean-shift, OPTICS **Hierarchical clustering:** MST, CURE, Chameleon, BIRCH, ROCK **Grid-based clustering:** STRING, WaveCluster, DCluster	**Partitioning Clustering:** – Relatively scalable and simple **Density-based clustering:** – Effectively discovering arbitrarily shaped clusters. – Resistant to noise and outliers compared to hierarchical or partitioning methods. – do not require a predetermined number of clusters **Hierarchical clustering:** Well suited for problems involving point connections (e.g. taxonomy trees), especially when dealing with datasets that exhibit vertical relationships. **Grid-based clustering:** – Effectively manage large datasets with high dimensionality. – Simple to implement and computationally efficient. – Provide good results for datasets with uniform density.	**Partitioning Clustering:** – User dependency to specify the number of clusters in advance. – Difficulty in identifying non-convex clusters, such as irregularly shaped or overlapping clusters. **Density-based clustering:** – High sensitivity to input parameter tuning. – Most density-based algorithms exhibit reduced efficiency with increased data dimensionality. **Hierarchical clustering:** – Faces challenges when dealing with noisy data in high-dimensional spaces. – Unable to rectify a split or merge decision once made. – Computationally expensive, especially with large datasets. **Grid-based clustering:** – Struggles with datasets that have variable densities or irregular shapes. – Sensitive to the choice of grid size or resolution – Unsuitable for data with outliers, which can significantly impact clustering results.

Table 3
Comparative table of missing data imputation techniques.

Category	Techniques	Strengths	Limitations
Traditional Methods [21]	Listwise Deletion, Pairwise Deletion	Simplicity, easy to implement	Leads to data loss, ineffective with large gaps
Basic Imputation Methods [22]	Mean Imputation, Regression Imputation	Preserves dataset size, simple to use	May introduce bias, ignores data uncertainty
Advanced Statistical Methods [23,24]	Multiple Imputation, Maximum Likelihood, EM Algorithm	Accounts for uncertainty, generates multiple datasets, refines estimates iteratively	Computationally intensive, sensitive to model assumptions
Machine Learning Methods [25–27]	KNN, Random Forest, MICE, XGBoost, Predictive Mean Matching (PMM), mixgb	Effective for complex data structures, scalable, robust	Can require large datasets, may be computationally expensive
Deep Learning Methods [28–31]	GAIN, Autoencoders, GAN-based methods	Highly accurate, superior performance in complex datasets	Struggles with small datasets, sample size constraints, high computational cost

Handling missing data has evolved significantly, beginning with traditional approaches that include Listwise Deletion and Pairwise Deletion. While these methods offer simplicity, they often lead to substantial data loss [21]. To preserve dataset integrity, imputation techniques have been developed, such as Mean Imputation, and Regression Imputation, which replace or predict missing values based on observed data [22]. Recent advancements have introduced sophisticated techniques like Multiple Imputation and Maximum Likelihood [23], which account for uncertainty by generating multiple datasets, along with the Expectation–Maximization (EM) Algorithm that iteratively refines missing data estimations [24].

Building on these methods, machine learning-based imputation techniques have emerged as powerful alternatives for handling missing data in complex datasets. These methods leverage data patterns to predict missing values effectively, improving imputation accuracy and accommodating more complex data structures [25]. Approaches like K-nearest neighbors (KNN), Random

Forest, and Multiple Imputation by Chained Equations (MICE) have been examined for their efficacy in addressing missing data challenges [25]. XGBoost, a gradient boosting technique, has been utilized to treat imputation as a supervised learning problem, demonstrating robustness against model misspecification [26]. To balance robustness and appropriate value generation, a novel Predictive Mean Matching (PMM) imputation procedure incorporating machine learning methods has been proposed [26]. Additionally, a scalable multiple imputation framework called mixgb, based on XGBoost, subsampling, and predictive mean matching, has been developed to automatically capture interactions and nonlinear relations while maintaining computational efficiency [27].

The evolution continues with deep learning-based imputation algorithms, which offer even more sophisticated solutions. Generative Adversarial Imputation Nets (GAIN) utilize a generator to impute missing values and a discriminator to ensure realism, outperforming traditional methods in various datasets [28]. Autoencoders and deep neural networks have been proposed as potential solutions for more accurate imputation, with genetic algorithms and fuzzy logic suggested as complementary techniques [29]. GAN-based methods have shown promise in handling different missing data types and fractions, demonstrating superior performance in terms of RMSE and FID compared to existing approaches [30]. However, a comprehensive study reveals that conventional methods like MICE and missForest may still be preferable for tabular data with limited sample sizes, as deep learning methods can struggle with certain missing data mechanisms and sample size constraints [31].

The choice of imputation method depends on several factors, including the size of the dataset, the nature of the missing data, and the computational resources available. Ultimately, selecting the appropriate imputation technique requires careful consideration of these elements to ensure the best results for data analysis.

3. **Data Deduplication** is a critical process in data management, addressing the challenge of identifying and eliminating duplicate records (Table 4). Traditional approaches often rely on similarity measures comparing attribute values, with matches determined by a threshold [36]. Fuzzy matching techniques, such as Levenshtein distance and Soundex, account for minor differences in data representation and phonetic similarities [37]. However, these methods may fall short in noisy data environments or large datasets. To improve accuracy and efficiency, researchers have explored iterative record linkage, which examines the context of tuples and their links to other records [36]. Indexing techniques have been developed to reduce the number of record pairs for comparison while maintaining high matching quality [38]. Recent advancements include adapting metric functional dependencies to improve detection accuracy by relaxing matching conditions on numeric values, resulting in significant improvements in precision and recall [39].

   In recent years, machine learning-based approaches have gained traction in automating and improving data deduplication. Supervised learning techniques like decision tree classification, random forests, and support vector machines (SVMs) have shown promise in learning complex patterns for identifying duplicate records. These approaches often incorporate various string comparison methods to enhance matching accuracy. Neural networks and boosted decision trees have shown equal or better performance compared to existing deduplication solutions [40]. Additionally, unsupervised learning methods like clustering algorithms for example K-means and DBSCAN can detect duplicates by grouping similar records without the need for labeled data. A machine learning framework using K-means clustering and similarity computations can improve the accuracy and efficiency of data deduplication in distributed systems with an optimal accuracy of 99.7% [41].

   Recent research has explored deep learning approaches for data deduplication in large-scale datasets. An Enhanced Deep Learning-based Record Deduplication (EDL-RD) framework using Long Short Term Memory (LSTM) variants has been proposed, which outperformed existing methods [42]. A hybrid BERT model combined with graph convolutional networks has been introduced for identifying near-duplicates based on semantic relationships between text documents [43]. Another method, SemDeDup, leverages pre-trained model embeddings to remove semantic duplicates, demonstrating improved efficiency and out-of-distribution performance [44]. Advanced neural architectures for approximate string matching and duplicate detection have also been explored, incorporating shortcut connections, self-attention mechanisms, and hard alignment attention [45]. These models outperformed traditional string similarity measures across various datasets. A GNN approach combining Graph Attention and LSTM was introduced to evaluate the correctness of duplicate groups in author name disambiguation [46]. A method integrating siamese and graph neural networks was developed to model and leverage entity relations for data integration, demonstrating superior performance over rule-based systems and non-graph-based deep learning approaches [47]. These studies highlight the effectiveness of deep learning techniques, including RNNs, transformers, and graph-based approaches, in handling complex data deduplication tasks and improving performance in large-scale datasets.

3.2. Normalization and standardization

1. **Normalization** also called "Min-Max Scaling" scales features in a dataset to a specified range, typically [0, 1], ensuring that data values fall within that range. This technique is widely used to adjust variables, making them more comparable. It replaces original values with new ones that fit within the defined range. Normalization is commonly applied in machine learning to enhance model accuracy by scaling raw data before it enters an algorithm, improving optimization and performance [48]. Where x is the original feature value. x_{min} and x_{max} are the minimum and maximum values of the feature in the dataset.

   $$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

   Normalization is beneficial for algorithms such as KNN and Neural Networks, where the distances between data points are significant. It ensures that all features contribute equally to model training. Moreover, normalization is particularly effective when you know the data does not follow a normal distribution [49].

2. **Standardization** also called "Z-score Scaling" transforms the data to have a mean of 0 and a standard deviation of 1, making features comparable across axes. It is particularly useful for models that assume normally distributed data, such as Linear Regression and SVM [50]. Where x is the original feature value. μ is the mean of the feature. and σ is the standard deviation of the feature.

   $$x_{std} = \frac{x - \mu}{\sigma}$$

   Standardization is more robust to outliers than normalization and works effectively with algorithms that assume normal distributions, such as Logistic Regression, SVM, and K-Means [50].

Table 4
Comparative table of data deduplication techniques.

Category	Techniques	Strengths	Limitations
Traditional Methods [36,37]	Similarity Measures (Threshold-based), Fuzzy Matching (Levenshtein, Soundex)	Simple and effective for small datasets, accounts for minor variations in data representation	Struggles with noisy data or large datasets, may lead to incorrect matches
Advanced Methods [36,38,39]	Iterative Record Linkage, Indexing Techniques, Metric Functional Dependencies	Improves accuracy and efficiency, reduces comparison pairs, boosts precision and recall	Computationally intensive, requires fine-tuning for large datasets
Machine Learning Methods [40,41]	Decision Trees, Random Forest, SVM, K-means, DBSCAN	Learns complex patterns, automates deduplication, highly accurate	Requires large training datasets, can be resource-intensive
Deep Learning Methods [42–47]	EDL-RD, BERT with GCN, SemDeDup, GNN-based Methods	Handles large-scale datasets, highly accurate for complex and semantic duplicates	High computational cost, requires significant data preprocessing

3.3. Feature engineering

Feature engineering is a big part of the overall data transformation process. Feature engineering mainly impacts the performance of models. Feature engineering is a term used in practice to improve the performance of machine learning algorithms. It deals specifically with creating new features or modifying existing features that will be used as input to machine learning algorithms [51]. Here are a few sub-domains contained within feature engineering that are worth mentioning (Table 5). Feature selection: This process is commonly referred to as deciding which features are important to make predictions. Feature extraction: This process deals with reducing the dimensionality of the dataset by transforming or combining features. This is done in order to make the data more useful for downstream processes like machine learning. Feature creation: During this stage, new features are created that may prove to be more indicative of a property possessed by a given observation [52].

1. **Feature Selection** is the process of identifying and selecting the most relevant features from the dataset that contribute to the predictive power of the model, while eliminating those that are redundant or irrelevant. Feature selection methods can be categorized into filters, wrappers, and embedded methods [53] Filter methods, such as Chi-square tests or correlation coefficients, evaluate the relevance of features based on their statistical properties. Wrapper methods, like Recursive Feature Elimination (RFE), iteratively test different feature subsets to find the most effective combination for the model. Embedded methods, such as those used in Lasso Regression or decision trees, perform feature selection during the model training process by penalizing less important features [54]. By focusing on the most significant features, models can be trained more efficiently and produce more reliable predictions.
2. **Feature Extraction** is a dimensionality reduction technique that transforms the original data into a lower-dimensional space while retaining the most informative characteristics of the data [54]. This process is particularly useful in scenarios where the original data is high-dimensional and sparse, which can lead to overfitting and increased computational complexity. Linear methods, such as Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA), reduce dimensionality by finding new components that maximize variance or class separability, respectively [55]. Non-linear methods, such as (t-SNE) and (ISOMAP), preserve the global or local structure of the data in the lower-dimensional representation, making them suitable for data with complex, non-linear relationships. By projecting data into a more compact feature space, feature extraction enhances the efficiency of machine learning algorithms and improves their generalization capabilities [55].
3. **Feature Creation** is a fundamental aspect of feature engineering. This process often involves the creation of polynomial features, which create new features from the existing ones by considering all polynomial combinations up to a specified degree [56]. For example, with two input features [a, b] and a degree of 2, the resulting feature set would include $[1, a, b, a^2, ab, b^2]$. This expansion captures both the individual effects and interactions of features, enabling models to learn more complex, non-linear relationships.
   Recent research has focused on developing automated approaches to feature engineering in machine learning. Chen et al. [57] proposed Neural Feature Search (NFS), a neural architecture that uses reinforcement learning to transform raw features through a series of functions, outperforming existing methods while mitigating feature explosion. Wang et al. [58] introduced LbR (Label based Regression), which mines correlations between feature pairs to generate informative features, demonstrating improved performance across various datasets and models. Eldeeb et al. [59] presented BigFeat, a scalable and interpretable framework that dynamically generates and selects expressive features. BigFeat showed superior performance compared to AutoFeat, improving the F1-Score of classifiers by an average of 4.59% and executing 22 times faster. These approaches aim to automate the feature engineering process, reducing the need for domain expertise and time-consuming manual efforts while enhancing model performance.

3.4. Encoding categorical data

Categorical variables often appear in datasets for classification and regression tasks, and they need to be encoded into numerical values before training [60] (Table 6). Encoding categorical data is a key step in machine learning, where categorical variables are converted into numerical formats that models can understand. One-hot encoding turns each category into a separate binary column (0 or 1), useful for variables without order. Frequency encoding replaces categories with how often they appear in the dataset. Binary encoding converts categories into binary code, reducing the number of columns compared to one-hot encoding. Mean encoding (or target encoding) assigns the average of the target variable to each category, though it can risk overfitting. Label encoding assigns a unique number to each category but might imply an order where none exists, while ordinal encoding is used when categories have a meaningful order. The choice of encoding method significantly impacts model performance, with different techniques being more suitable for specific model types and dataset characteristics [61].

Traditional methods like one-hot encoding can lead to high dimensionality and memory inefficiency [62]. Alternative approaches include label, count, and target encoding [65]. Lopez-Arevalo et al. [62]

Table 5
Comparative table of feature engineering techniques.

Category	Techniques	Strengths	Limitations
Feature Selection [53,54]	Filter Methods (Chi-Square, Correlation), Wrapper Methods (RFE), Embedded Methods (Lasso, Decision Trees)	Enhances model efficiency, removes irrelevant features, improves prediction accuracy	Can be computationally expensive, especially with wrapper methods
Feature Extraction [54,55]	PCA, LDA, t-SNE, ISOMAP	Reduces dimensionality, addresses overfitting, and enhances generalization	Non-linear methods like t-SNE may be slow and scale poorly with large datasets
Feature Creation [56]	Polynomial Features	Captures non-linear relationships, enhances model performance	Can lead to feature explosion, increasing computational burden
Automated Feature Engineering [57–59]	Neural Feature Search (NFS), LbR, BigFeat	Reduces manual effort, improves model performance, scalable and interpretable	Requires sophisticated algorithms, may struggle with complex feature interactions

Table 6
Comparative table of categorical encoding techniques.

Encoding method	Description	Strengths	Limitations
One-Hot Encoding [60,62]	Creates a binary column for each category	Works well for unordered categorical variables	Can lead to high dimensionality and memory inefficiency
Label Encoding [60,61]	Assigns a unique integer to each category	Simple and easy to implement	May imply an order where none exists
Ordinal Encoding [61,63]	Assigns numbers based on order of categories	Useful when the categories have a meaningful order	Not suitable for unordered variables, can introduce bias
Frequency Encoding [60]	Replaces categories with their frequency in the dataset	Reduces dimensionality compared to one-hot encoding	Not suitable for models sensitive to value distribution
Mean (Target) Encoding [60]	Encodes categories based on target variable averages	Useful in capturing category relevance	Risk of overfitting, especially on small datasets
Binary Encoding [60]	Converts categories to binary codes	Reduces number of columns compared to one-hot encoding	Less interpretable and may introduce noise
Shannon's Theory-based Encoding [62]	Novel encoding using information theory	Outperforms one-hot in memory efficiency	Limited studies on its applicability across models
Sample Empirical Probability (SEP) [64]	Encodes based on sample probabilities	More sensible results for neural networks compared to PEL	Not widely tested outside specific neural network applications

proposed a novel encoding method using Shannon's Theory, which outperformed one-hot and feature-hashing in memory efficiency. In software development effort estimation, one-hot encoding proved suitable for various machine learning estimators [65]. For neural networks, Sample Empirical Probability (SEP) encoding was found to be superior to Population Evidence Likelihood (PEL) encoding, producing more sensible results [64]. However, the impact of categorical encoding on model interpretability remains understudied. Hakkoum et al. [63] found that both ordinal and one-hot encoding performed poorly in terms of interpretability for a multilayer perceptron in breast cancer classification, with a slight preference for one-hot encoding.

3.5. Data augmentation

Data augmentation is a technique used to artificially increase the size and diversity of training datasets for machine learning models [66, 67]. It addresses challenges such as limited training data, overfitting, and privacy concerns [67]. Various methods are employed across different domains (Table 7), including image processing, natural language processing, and signal data [66].

In tabular data, challenges like imbalanced learning and small datasets have led to increased attention towards data augmentation techniques. Various approaches have been proposed to address these issues. Machado et al. [73] benchmarked different techniques to identify dataset characteristics that yield better performance when synthesizing samples. Fang et al. [70] introduced SDAT, a semi-supervised learning method using a variational auto-encoder for data augmentation in latent space. Khan et al. [71] developed TabAug, which creates structural changes in table images through row and column manipulation, showing improvements in table structure recognition. Onishi et al. [72] proposed Mask Token Replacement (MTR), a novel augmentation method for Transformer-based models that replaces mask tokens with portions of tokenized columns. Their experiments across 13 datasets demonstrated competitive performance against existing methods. These studies highlight the importance of tailored augmentation techniques for tabular data, as traditional image-based methods may not be effective in this domain [71]. SMOTE introduced by Apostolopoulos et al. [68] is a popular method for generating synthetic samples in the minority class. While SMOTE and its variants have shown effectiveness, Temraz et al. [69] also unveiled a novel Counterfactual Augmentation (CFA) method, which demonstrates competitive results by adaptively combining existing instances using actual feature values. Overall, data augmentation techniques enhance model robustness, accuracy, and generalization across different data types and machine learning tasks [66].

3.6. Discretization

Discretization is a crucial preprocessing technique that transforms continuous attributes into discrete intervals, enabling various machine learning algorithms and simplifying data interpretation [74]. Common

Table 7
Comparative table of data augmentation techniques.

Augmentation method	Description	Strengths	Limitations
SMOTE [68,69]	Generates synthetic samples for the minority class by interpolating between existing instances	Effective in addressing class imbalance	May create overlapping classes or introduce noise in the data
CFA [69]	Combines existing instances adaptively using actual feature values to generate new samples	Shows competitive results in tabular datasets	Requires careful feature selection and can be computationally expensive
SDAT [70]	Uses a variational auto-encoder to augment data in latent space	Improves model generalization and works well with small datasets	Complex to implement, particularly for large-scale datasets
TabAug [71]	Applies structural changes to table images through row and column manipulation to improve table structure recognition	Enhances recognition of tabular structures	Limited to table-structured data and may not generalize well to other domains
MTR [72]	Replaces masked tokens with tokenized column portions for Transformer-based models	Improves performance in Transformer models, especially for tabular datasets	Requires large pre-trained models and might not be suitable for smaller tasks

Table 8
Comparative table of discretization techniques.

Discretization method	Description	Strengths	Limitations
EWI	Divides the range of a continuous variable into intervals of equal size	Simple to implement and interpret	May result in imbalanced bins, affecting model performance
EFI	Divides the data such that each bin contains an equal number of data points	Ensures balanced representation of data points in each bin	Bin widths can vary widely, leading to non-uniform intervals
KMC	Uses a clustering algorithm to group continuous data into K clusters, forming bins	Reflects natural groupings in the data, often producing more meaningful intervals	Requires specifying the number of clusters, which can be challenging

methods, such as equal width intervals (EWI), equal frequency intervals (EFI), and K-means clustering (KMC), have been shown to improve the performance of classification models like decision trees [75] (Table 8).

Equal Width Intervals (EWI) divides the range of a continuous variable into intervals of equal size. For example, if the data ranges from 0 to 100 and you create five bins, each bin will cover a range of 20 units (0-20, 21-40, and so on). While this method ensures uniform interval sizes, it may result in some bins having many data points and others having few, leading to imbalanced splits in decision trees.

Equal Frequency Intervals (EFI), by contrast, ensures that each bin contains an equal number of data points, though the width of the bins can vary. This method leads to more balanced splits in decision trees by ensuring that each bin represents an equally important portion of the dataset. For instance, if there are 100 data points and five bins, each bin will contain 20 points, but the range of values in each bin may differ.

K-means Clustering (KMC) uses a clustering algorithm to group continuous data into K clusters, with each cluster forming a bin. This method identifies natural groupings in the data and creates bins that reflect these patterns. KMC often produces more meaningful intervals than EWI or EFI, as the bins are based on underlying data structures rather than arbitrary cutoffs.

Discretization is especially important for algorithms requiring discrete inputs, such as association rules, Bayesian networks, and induction rules [76]. It offers several benefits, including easier human interpretation, faster computations, and the potential to uncover non-linear relationships [76]. Choosing the right discretization method depends on the learning task and dataset characteristics, and ongoing research aims to find optimal techniques for various analytical tasks, such as association analysis [77].

3.7. Data aggregation

Data aggregation is a crucial step in data transformation that involves combining and summarizing information to make it more suitable for analysis. There are generally two types of aggregation: traditional aggregation methods and recent, more advanced aggregation techniques. Traditional methods are typically focused on basic statistical summaries and simple grouping [78], while recent techniques are designed to handle large-scale, real-time data and incorporate machine learning to uncover complex patterns [79]. These two types of aggregation each have their strengths, depending on the scale of the data and the goals of the analysis.

Traditional approaches to data aggregation primarily involve straightforward statistical summaries or grouping techniques that have been in use for many years. One such method is SQL-based aggregation, which is commonly used in relational databases. Functions like SUM(), AVG(), COUNT(), GROUP BY, and JOIN operations allow for the aggregation of data [80]. However, this approach lacks the sophistication needed to handle more complex patterns and can become slow with large datasets.

In programming environments like Python's pandas and R, groupby operations are commonly used for grouping data by specific columns and performing aggregations such as summing, averaging, or counting. For example, in pandas, df.groupby(['column']).agg('mean') calculates group-based means. However, while these operations are flexible, they can be inefficient for large-scale data processing, as dataframes in these environments often face performance issues when handling large datasets [81].

Another traditional approach is OLAP (Online Analytical processing), a crucial technology for multi-dimensional data analysis, commonly used in data warehousing. It enables operations like slicing, dicing, roll-up, and drill-down, allowing businesses to extract valuable

insights from large datasets [82]. However, traditional OLAP systems often require heavy infrastructure and are less suited for real-time, fast-changing data. Furthermore, they face scalability challenges when dealing with big data. To address these limitations, solutions like OLAP on Hadoop by Indexing (OOHI) have been proposed. This approach simplifies the multi-dimensional model using techniques like dimension encoding, cube segmentation, and MapReduce-based indexing, improving data storage efficiency, processing speed, and parallelism in OLAP operations for big data analytics [83]. These advancements make OLAP more suitable for modern business intelligence applications.

With advancements in technology and machine learning, recent aggregation methods focus more on scalability, real-time data processing, and predictive aggregation. One such method is stream aggregation, which has become essential with the rise of real-time. Scotty, an open-source operator, offers general stream slicing and user-defined aggregation functions, outperforming alternative solutions [84]. Multi-dimensional aggregation of sensor data streams requires hierarchical grouping, multiple parallel hierarchies, runtime reconfiguration, and scalability [85]. Apache Flink, an open-source system, provides expressive, declarative, and efficient data analysis for both batch and streaming data, featuring a distributed dataflow runtime, flexible windowing semantics, and stateful, fault-tolerant computations [86].

Distributed data aggregation has emerged as a powerful approach for processing massive datasets. MapReduce and Hadoop, key frameworks in this domain, enable efficient parallel processing of large-scale data across multiple clusters [87]. These systems utilize distributed file systems to manage and analyze data across numerous processors, offering scalability and fault tolerance [88]. While MapReduce is widely used, studies have shown that its performance can vary depending on the specific computation and execution plan [89]. The effectiveness of user-defined aggregations in distributed systems is influenced by factors such as language integration and programming interfaces [89]. As data dimensionality increases, addressing the curse of dimensionality becomes crucial. Techniques like Dimension Aggregation Approximation (DAA) have been proposed to reduce both communication and computation costs in high-dimensional similarity joins using MapReduce [90].

To improve aggregation performance on very large datasets, approximate aggregation techniques such as sketches, histograms, and sampling are often used. HyperLogLog is a near-optimal algorithm for estimating cardinality with high accuracy using minimal memory, making it suitable for datasets exceeding 10ϑ elements [91]. Building on this, Ting et al. proposed combining CountMin and HyperLogLog sketches to compute millions of distinct count aggregations in a single pass, offering theoretical guarantees and tight error estimates [92]. These methods are particularly useful for applications in networking, databases, and real-time reporting. Online aggregation, another approach, continuously retrieves samples from databases to generate approximate results, allowing users to terminate processing when satisfied with the quality [93]. This technique is cost-effective for large-scale datasets and relies on sampling approaches and estimation models. These probabilistic algorithms enable faster aggregation and real-time performance, albeit with a slight trade-off in accuracy.

Machine learning has also introduced machine learning-based aggregation techniques that involve discovering patterns or relationships in the data. K-means clustering, one of the most popular algorithms, groups similar data points into clusters based on feature similarity [94]. Hierarchical clustering, another common approach, can be either agglomerative or divisive [95]. These unsupervised learning methods are particularly useful in scenarios where labeled data is unavailable [96]. In wireless sensor networks, machine learning algorithms like swarm intelligence and neural networks have been applied to clustering and data aggregation, reducing energy consumption by decreasing data transmission [97]. The choice of clustering technique depends on the specific application, with each method having its own advantages and disadvantages [95].

Finally, graph aggregation is a crucial technique for summarizing and analyzing complex network structures, particularly in social networks and recommendation systems. It involves combining multiple input graphs to produce a single output graph that represents a compromise or consensus [98]. Various methods have been developed for graph aggregation, including statistical approaches and database-style operations like SNAP and k-SNAP, which allow users to control summary resolution and navigate through different levels of detail [99]. These techniques are essential for understanding large graph datasets and uncovering hidden relationships between entities [100] (Table 9).

4. Applications and challenges in modern data transformation

Data transformation is an essential part of preparing datasets for analysis, but it is not without its challenges. As datasets grow in complexity and size, the task of transforming data while ensuring its accuracy, consistency, and scalability becomes increasingly difficult [101]. Moreover, modern data environments—where data is generated in real-time, sourced from various systems for example, the data comes from many different sources, including e-commerce, credit scoring, healthcare, and others [102]. A solid grasp of data analysis, along with fundamental statistical and machine learning techniques, is crucial for effectively implementing data transformations. This section highlights key applications and challenges in data transformation, as well as best practices for addressing them.

4.1. Applications

E-commerce Industry, data transformation plays a vital role in enhancing the quality of services and boosting revenues in the e-commerce industry. A key use case involves analyzing customer behavior and preferences to improve customer profiles. Advanced analytics offer numerous benefits, such as real-time responses, personalized product offers, and inventory optimization. This process helps retailers make better product recommendations, forecast sales more accurately, and reduce inventory costs. Additionally, data transformation improves business intelligence systems by facilitating cross-data migration and enabling more efficient decision-making across business functions [103].

Healthcare Sector, data transformation holds significant potential in healthcare, where vast amounts of patient data, often measured in petabytes, need to be stored, managed, and retrieved across various formats. Due to regulatory restrictions, healthcare data cannot always be stored in the cloud, but new infrastructures allow health systems to manage diverse data. Transformed data supports clinical decision-making, treatment planning, and personalized care, ultimately leading to better healthcare outcomes and resource efficiency [104].

Financial Services, in the highly regulated and volatile financial sector, information asymmetry poses a significant challenge. Verifiable data is essential for decision-making, especially in areas like risk management and compliance, where the growing volume and complexity of data make meeting these needs difficult [105]. Financial institutions have adopted big data analytics to address these challenges but face unique issues. These include the complexity of regulations such as Know Your Customer (KYC) and anti-money laundering (AML) rules, and the data silos that prevent a comprehensive view of clients' financial histories. One bank addressed this by centralizing its data, allowing faster access for queries and fraud analytics [106]. Another financial services company processed 730 million credit card transactions to provide insights into foreign transaction behavior, aiding its member banks in understanding cardholder activity overseas.

Table 9
Comparative table of data aggregation techniques.

Aggregation method	Description	Strengths	Limitations
SQL-based Aggregation [80]	Uses SQL functions like SUM(), AVG(), COUNT(), and GROUP BY for simple data summarization	Easy to implement and widely used in relational databases	Lacks sophistication for complex patterns and slows down with large datasets
Groupby Operations in Python/R [81]	Performs aggregation based on groupings of data, e.g., using pandas or R	Flexible and allows various types of aggregation	Performance issues arise when dealing with large datasets
OLAP [82,83]	Enables multi-dimensional analysis of data, commonly used in data warehousing	Supports slicing, dicing, roll-up, and drill-down for in-depth analysis	Requires heavy infrastructure and struggles with real-time or big data processing
Stream Aggregation [84,85]	Processes real-time data streams with operators like Scotty for slicing and aggregation	Handles real-time, scalable data processing efficiently	Complex to set up and manage, particularly for large-scale applications
Distributed Data Aggregation [87–90]	Processes large-scale data in parallel across clusters, using distributed file systems	Highly scalable and fault-tolerant for massive datasets	Performance depends on execution plan, and handling complex computations can be challenging
Approximate Aggregation [91–93]	Uses algorithms like sketches, histograms, or sampling for approximate aggregation	Fast, memory-efficient, and suitable for large datasets	Trade-off in accuracy, but still offers high-quality estimations
Machine Learning-based Aggregation [94–97]	Employs algorithms like K-means or hierarchical clustering to aggregate data based on patterns	Unsupervised learning techniques can uncover complex relationships in data	Performance and accuracy depend on algorithm and application; may require high computational power
Graph Aggregation [98–100]	Summarizes complex networks by combining input graphs	Enables detailed analysis of large graph datasets and hidden relationships	Managing and analyzing massive graphs can be computationally expensive

4.2. Challenges

Data Quality Issues are one of the biggest challenges in data transformation. Common data quality problems such as missing data, inaccuracies, duplications, inconsistencies, and outliers can have severe consequences if analytical outcomes are based on uncleaned raw data. For instance, inaccurate or missing values in a dataset can lead to incorrect conclusions in analytical models, especially in survey analysis or machine learning predictions. Moreover, duplicates and anomalies can skew results, leading to biased interpretations. Ensuring data quality is critical, and various techniques like data validation, profiling, and automated quality reports can help. By implementing robust data quality controls, organizations can ensure that data is both accurate and reliable, ultimately leading to better decision-making [107,108].

Scalability Challenges, As datasets grow in size and complexity, scalability becomes a significant concern. Large datasets often require more time and computational resources for transformation, which can slow down the entire analytics process. Additionally, marshaling large volumes of data for processing, especially in real-time applications, can overwhelm traditional systems. To address these challenges, cloud-based tools and distributed computing systems that leverage parallel processing are frequently employed. These systems help process large datasets more efficiently by breaking them down into smaller, manageable units. Designing scalable transformation workflows and adopting distributed processing frameworks such as Hadoop or Apache Spark can ensure that even the largest datasets are transformed in a timely and efficient manner [109].

Automation and Workflow Management, In modern data environments, automation plays a critical role in streamlining the transformation process. Manually transforming data, especially when dealing with large datasets, can be time-consuming and prone to errors. Automating repetitive tasks in data transformation not only increases efficiency but also reduces the chances of human error. Workflow automation tools allow for the orchestration of various transformation steps, from data extraction and cleaning to validation and loading into analytical systems. These tools help ensure that data transformation processes run smoothly and consistently, even when handling complex, multi-stage workflows. Adopting automation best practices, such as monitoring for errors and implementing fail-safe mechanisms, can further improve the reliability and scalability of data transformation efforts [110].

Data Integration poses a major challenge, especially when data comes from diverse sources like relational databases, APIs, spreadsheets, or even unstructured formats such as text and images. Each of these sources may have different data structures, schemas, and formats, making it difficult to consolidate them into a unified dataset. Harmonizing these disparate datasets without losing essential information requires effective data integration strategies, including schema mapping and ETL (Extract, Transform, Load) processes. ETL tools can help standardize the format and structure of incoming data, enabling a more coherent analysis. Data integration is essential not only for merging datasets but also for ensuring that analytical models built on this data provide accurate and actionable insights [111].

Maintaining Data Consistency, As data undergoes multiple transformations — such as filtering, aggregation, or normalization — there is a risk of introducing inconsistencies between the original data and its transformed versions. Inconsistencies can lead to inaccurate analysis, which may result in faulty business decisions. Implementing version control mechanisms and tracking data lineage can help maintain consistency. By documenting each transformation step and its effect on the data, organizations can ensure that the

transformed datasets accurately reflect the original intent and that errors are minimized [112].

Real-Time Data Transformation, With the rise of real-time applications in industries such as IoT, finance, and e-commerce, the demand for real-time data transformation has surged. Processing data as it is generated — rather than relying on traditional batch processing — enables quicker decision-making. However, transforming data in real-time presents unique challenges, including maintaining low-latency processing, handling high-velocity data streams, and ensuring accurate transformations on-the-fly. Stream processing frameworks like Apache Kafka and Apache Flink have become popular solutions for real-time data transformation, enabling organizations to handle continuous data streams and perform operations like windowed aggregation and filtering in real-time [113].

Data Security and Privacy are critical considerations in any transformation process, especially when handling sensitive or personal data. During data transformation, there is a risk of exposing sensitive information or violating privacy regulations like GDPR or HIPAA. Ensuring that data remains secure throughout its transformation requires implementing encryption, anonymization, or tokenization techniques. Additionally, using role-based access control (RBAC) can ensure that only authorized individuals have access to sensitive data during the transformation process. Organizations must also ensure that they comply with data privacy regulations, both during and after transformation, to avoid legal and financial penalties [114].

Handling Unstructured Data, such as text, images, or videos, adds an additional layer of complexity to data transformation. While structured data like tables can be easily transformed using traditional methods, unstructured data requires specialized techniques. Natural language processing (NLP) for text and image processing frameworks for visual data are some common methods used to transform unstructured data into a format that can be analyzed. Integrating unstructured data with structured data for unified analysis can be challenging but is necessary for generating comprehensive insights. Hybrid data transformation frameworks that can handle both structured and unstructured data are becoming increasingly important in modern analytics [115].

Ensuring Data Accuracy Post-Transformation is critical to achieving valid analytical results. During transformation, errors such as incorrect mappings or rounding issues can occur, potentially skewing results. This makes it essential to validate transformed data against the original dataset. Organizations should implement data validation steps throughout the transformation process to ensure that no errors are introduced and that the transformed data remains accurate and useful for analysis. Regular audits and quality assessments can also help detect inaccuracies before they impact downstream analyses [116].

Cost and Resource Management, the resource demands for large-scale data transformations can be considerable, particularly when dealing with big data. Computational resources such as CPU, memory, and storage are necessary to handle large datasets, and the costs associated with these resources can escalate quickly. Cloud-based solutions offer scalability, allowing organizations to pay for only the resources they use, which helps in managing costs. However, organizations must carefully balance cost considerations with the need for robust data transformation processes, ensuring that budget constraints do not compromise the quality or scalability of the transformation efforts [117].

By addressing these challenges through best practices, data practitioners can ensure that their transformation processes are both effective and efficient, enabling them to derive maximum value from their datasets.

5. The role of domain-specific languages and general-purpose languages in data transformation

In the rapidly evolving landscape of data management and analysis, the choice of programming language plays a crucial role in the efficiency and effectiveness of data transformation processes. Two primary categories of programming languages are often employed: Domain-Specific Languages (DSLs) and General-Purpose Languages (GPLs). Each has its unique characteristics, advantages, and challenges that make them suitable for different aspects of data transformation [118, 119].

Domain-Specific Languages: are specialized programming languages tailored to particular application domains, offering more natural and efficient ways to express solutions compared to general-purpose languages [120]. Their syntax and functionality are optimized for specific use cases, which makes them particularly effective in certain contexts. For instance, SQL (Structured Query Language) is a DSL used for managing and querying relational databases. It provides a set of commands and functions that allow users to retrieve, manipulate, and transform data efficiently. The simplicity and specificity of SQL enable data analysts and database administrators to perform complex queries without needing extensive programming knowledge.

Another example of a DSL is XSLT (Extensible Stylesheet Language Transformations), which is used primarily for transforming XML documents into different formats. XSLT allows for the application of rules and templates to XML data, facilitating the extraction and restructuring of information in a way that is both powerful and intuitive for users familiar with XML. The advantages of using DSLs in data transformation include:

- DSLs are optimized for specific tasks, allowing for faster execution and reduced complexity in data manipulation [120].
- The focused syntax of DSLs often makes them easier to learn and use, especially for domain experts who may not have a strong programming background [121].
- DSLs provide clear and concise expressions for specific operations, making it easier to understand and maintain code [122].
  However, DSLs also come with challenges. Their limited scope can restrict their applicability in broader data transformation scenarios. Integrating DSLs with other systems or programming languages can be complex, particularly if the DSL lacks robust interoperability features. Additionally, the smaller communities surrounding many DSLs may result in fewer resources and libraries available for users [123].

General-Purpose Languages: in contrast, General-Purpose Languages (GPLs) are designed to be versatile and applicable across a wide range of programming tasks. Languages such as Python, Java, and C++ offer extensive libraries and frameworks that facilitate various aspects of data transformation [124], from data cleaning and preprocessing to complex analysis and visualization.

Python, in particular, has gained immense popularity in the data science community due to its simplicity and the richness of its libraries, such as Pandas and NumPy. These libraries provide powerful tools for data manipulation, enabling users to perform complex transformations with relatively simple code. The flexibility of GPLs allows developers to create custom solutions tailored to specific data transformation needs, making them suitable for comprehensive projects that require integration with multiple data sources and formats. The benefits of using GPLs for data transformation include:

- GPLs can be adapted for various tasks, allowing for the implementation of custom algorithms and processes [125].
- The extensive libraries available for General Purpose Languages (GPLs) offer a rich array of resources for managing various data types and formats, facilitating the implementation of complex transformations. This is particularly evident in fields such as bioinformatics [126], digital libraries [127], and genomics [128].

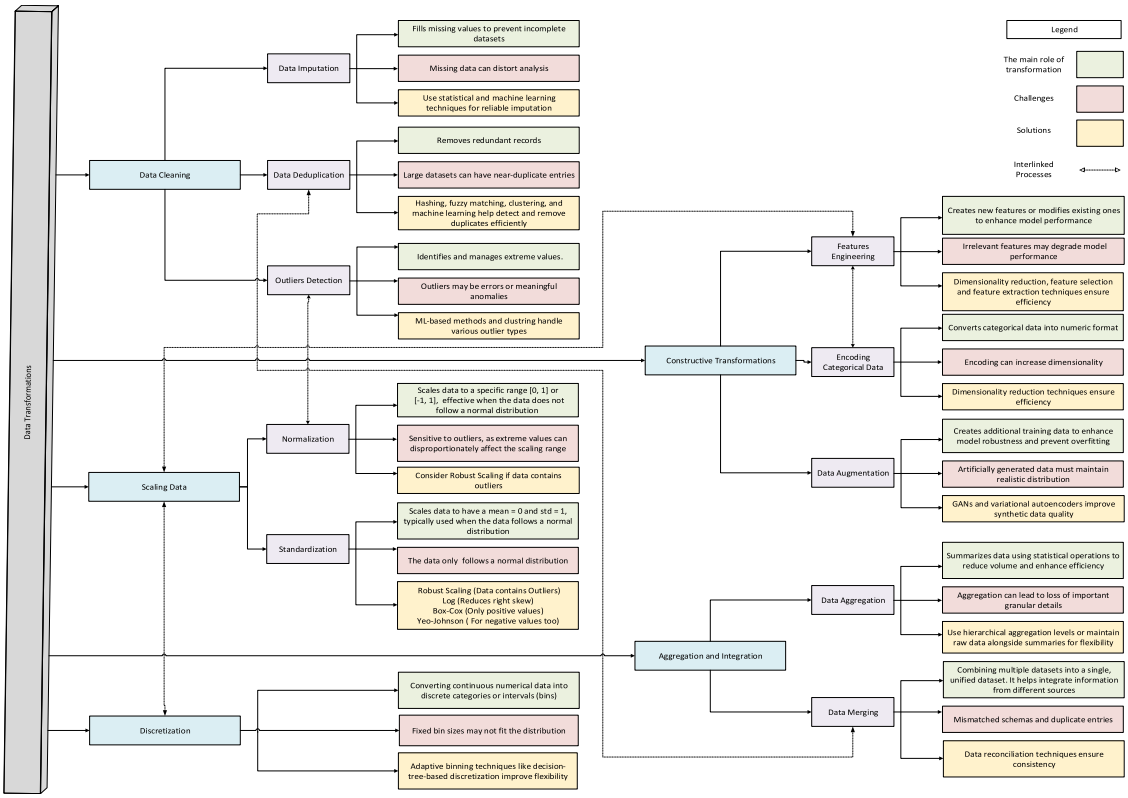

Fig. 1. Integrated Framework for Data Transformation: Ensuring Quality and Efficiency.

- Larger communities surrounding GPLs result in more resources, documentation, and support, making it easier for users to troubleshoot issues and learn best practices [129].

Nevertheless, GPLs also face challenges. The broad feature set can lead to a steeper learning curve, particularly for domain experts who may be less familiar with programming concepts. Additionally, the performance of GPLs may not be optimized for specific tasks, potentially leading to inefficiencies in data processing when compared to DSLs [130].

To summarize, both Domain-Specific Languages and General-Purpose Languages play significant roles in data transformation, each offering distinct advantages and challenges. DSLs excel in specific tasks with tailored syntax and optimized performance, making them ideal for focused data manipulation within particular domains. On the other hand, GPLs provide the flexibility and power needed for complex and varied data transformation projects, accommodating a broader range of applications. Choosing the right language for data transformation ultimately depends on the specific requirements of the project at hand. For tasks that require specialized operations within a defined domain, DSLs may be the best choice. Conversely, for comprehensive data transformation projects that demand flexibility and integration across multiple data sources, GPLs are likely to be more suitable. Understanding the strengths and limitations of both types of languages will empower practitioners to make informed decisions, enhancing their ability to transform and analyze data effectively.

6. Synthesis and discussions

Data transformation plays a crucial role in ensuring that raw data becomes structured, meaningful, and usable for analytical and machine learning tasks. In this study, we have explored various transformation techniques categorized into five key areas: Data Cleaning, Constructive Transformations, Scaling, Aggregation and Integration, and Discretization. Each category has its specific purpose, challenges, and solutions, as highlighted in previous sections and summarized in the schema (Fig. 1). Arrows between transformations indicate dependencies. Some processes must be performed before others to ensure consistency and efficiency.

6.1. Key observations and insights

Data cleaning is the first critical phase of transformation, ensuring that errors, inconsistencies, and missing values do not compromise downstream processes. Techniques like imputation, deduplication, and outlier detection significantly impact the reliability of subsequent transformations.

The use of machine learning methods, such as clustering for deduplication or ML-based imputation, enhances efficiency.

Feature engineering, categorical encoding, and data augmentation contribute to refining data representations. While feature engineering can enhance model performance, it must be done carefully to avoid introducing irrelevant features. Similarly, categorical encoding techniques, though necessary for many machine learning algorithms, can increase dimensionality, requiring further processing through dimensionality reduction techniques. Data augmentation, on the other hand, generates synthetic data to enhance model robustness and prevent overfitting. However, ensuring that augmented data maintains realistic distributions is crucial to preserving data integrity.

Scaling ensures that numerical values remain within comparable ranges, facilitating stable and accurate learning for machine learning models. While normalization is suitable for non-normally distributed data, standardization works well for normally distributed data. However, special attention is required when handling outliers, where robust scaling methods such as the median-based robust scaler offer more resilience. Additionally, log transformation helps reduce right skewness, while Box–Cox and Yeo–Johnson transformations handle non-normal distributions effectively, including cases with negative values. Choosing the appropriate scaling method depends on the data distribution and

the presence of outliers, ensuring that transformed features contribute meaningfully to model performance.

Combining multiple datasets through aggregation or merging can improve data completeness and analysis efficiency. However, challenges such as schema mismatches and duplicate entries require careful reconciliation techniques. Hierarchical aggregation approaches provide flexibility in summarizing data while preserving important granularity.

Discretization is essential for converting continuous data into categorical representations, which can improve interpretability and enable certain model types. However, fixed bin sizes may not always be optimal, and adaptive binning techniques like decision-tree-based discretization provide more flexibility.

6.2. Interdependencies between transformations

One key observation from our study is that transformations do not operate in isolation; they are highly interdependent. The following relationships are particularly important:

1. Feature Engineering - Scaling: Many feature extraction techniques, such as PCA, require standardized input for optimal performance.
2. Outlier Detection - Scaling: Normalization and standardization are sensitive to outliers, making robust scaling methods essential when extreme values exist.
3. Encoding Categorical Data - Dimensionality Reduction: One-hot encoding can increase dimensionality, necessitating feature selection or extraction techniques to maintain efficiency.
4. Data Merging - Deduplication: Merging datasets often leads to near-duplicate records, requiring post-processing for consistency.
5. Discretization - Scaling: The way continuous data is scaled influences binning strategies, affecting how well discretized categories represent the underlying distribution.

The effectiveness of data transformation lies in choosing the right combination of techniques based on the dataset characteristics and the specific analytical or machine learning goals. Future work could explore automated transformation workflows that dynamically adjust based on data properties, further optimizing preprocessing pipelines.

6.3. Conclusion

Data transformation is a fundamental step in the data preprocessing pipeline, ensuring that raw data becomes structured, meaningful, and suitable for analysis. This article has provided a comprehensive survey of data transformation, starting with its definition and importance, followed by an exploration of various transformation types, techniques, and tools. By studying these methods, we highlighted their roles in enhancing data quality, improving machine learning model performance, and addressing key challenges such as noise, missing values, and inconsistencies.

Despite its advantages, data transformation presents several challenges that must be carefully managed. Data quality issues remain a primary concern, as inconsistencies, outliers, and missing values can compromise the effectiveness of transformations. Additionally, data security and privacy are critical in sensitive domains, requiring careful handling to prevent unauthorized access and data breaches. Data integration across multiple sources introduces complexities related to schema mismatches, redundancy, and standardization. Finally, cost and resource management pose significant constraints, particularly when dealing with large-scale datasets requiring extensive computational resources. Addressing these challenges is crucial for designing robust and scalable transformation pipelines.

While this study lays a strong foundation for understanding data transformation, our future research will focus on developing a complete data-cleaning workflow that integrates transformation techniques for large-scale datasets. This upcoming work will emphasize automation, scalability, and optimization, leveraging machine learning-based approaches to enhance efficiency. The goal is to design a comprehensive and adaptive preprocessing pipeline that not only ensures data quality but also optimizes resource utilization for high-dimensional and heterogeneous data. By applying this workflow to real-world datasets, we aim to bridge the gap between theoretical transformation methodologies and practical implementations in large-scale data environments.

As data ecosystems evolve, the future of data transformation will be influenced by the increasing demands for real-time analytics, the rise of unstructured and multimodal data, and the growing reliance on AI-driven automation. Techniques such as transformer-based models for data wrangling, automated feature engineering, and low-code/no-code transformation platforms are expected to gain traction, lowering barriers for non-technical users while maintaining high transformation quality. The shift towards data-centric AI focusing on enhancing data quality rather than merely improving models will further underscore the need for sophisticated transformation pipelines.

CRediT authorship contribution statement

Sanae Borrohou: Writing – original draft. **Rachida Fissoune:** Supervision. **Hassan Badir:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Cheng Fan, Meiling Chen, Xinghua Wang, et al., A review on data preprocessing techniques toward efficient and reliable knowledge discovery from building operational data, Front. Energy Res. 9 (2021) 652801.
[2] Zhijian Wang, Wenlei Zhao, Wenhua Du, et al., Data-driven fault diagnosis method based on the conversion of erosion operation signals into images and convolutional neural networks, Process. Saf. Environ. Prot. 149 (2021) 591–601.
[3] Abdalwali Lutfi, Adi Alsyouf, Mohammed Amin Almaiah, et al., Factors influencing the adoption of big data analytics in the digital transformation era: Case study of Jordanian SMEs, Sustainability 14 (3) (2022) 1802.
[4] Iqbal H. Sarker, Data science and analytics: an overview from data-driven smart computing, decision-making and applications perspective, SN Comput. Sci. 2 (5) (2021) 377.
[5] Nadine Côrte-real, Pedro Ruivo, Tiago et Oliveira, Leveraging internet of things and big data analytics initiatives in European and American firms: Is data quality a way to extract business value? Inf. Manag. 57 (1) (2020) 103141.
[6] Swen et Nadkarni, Reinhard Prügl, Digital transformation: a review, synthesis and opportunities for future research, Manag. Rev. Q. 71 (2021) 233–341.
[7] Hamed Taherdoost, Data collection methods and tools for research; a step-by-step guide to choose data collection technique for academic and business research projects, Int. J. Acad. Res. Manag. (IJARM) 10 (1) (2021) 10–38.
[8] Sascha Kraus, Paul Jones, Norbert Kailer, et al., Digital transformation: An overview of the current state of the art of research, Sage Open 11 (3) (2021) 21582440211047576.
[9] Jingran Wang, Yi Liu, Peigong Li, et al., Overview of data quality: Examining the dimensions, antecedents, and impacts of data quality, J. Knowl. Econ. 15 (1) (2024) 1159–1178.
[10] Mengjia Wu, Dilek Cetindamar Kozanoglu, Chao Min, et al., Unraveling the capabilities that enable digital transformation: A data-driven methodology and the case of artificial intelligence, Adv. Eng. Informatics 50 (2021) 101368.
[11] Steven Euijong Whang, Yuji Roh, Hwanjun Song, et al., Data collection and quality challenges in deep learning: A data-centric ai perspective. The.
[12] Kai Xiong, Siwei Fu, Guoming Ding, et al., Visualizing the scripts of data wrangling with SOMNUS, IEEE Trans. Vis. Comput. Graphics 29 (6) (2022) 2950–2964.
[13] Sean Kandel, Andreas Paepcke, Joseph Hellerstein, et al., Wrangler: Interactive visual specification of data transformation scripts, Proc. the Sigchi Conf. Hum. Factors Comput. Syst. (2011) 3363–3372.

[14] Yeye He, Xu Chu, Kris Ganjam, et al., Transform-data-by-example (TDE) an extensible search engine for data transformations, Proc. the VLDB Endow. 11 (10) (2018) 1165–1177.

[15] Jason Brownlee, Data preparation for machine learning: data cleaning, feature selection, and data transforms in Python, Mach. Learn. Mastery (2020).

[16] Chen-Hao Chao, Wei-Fang Sun, Yen-Chang Hsu, et al., Training energy-based normalizing flow with score-matching objectives, Adv. Neural Inf. Process. Syst. (2024) 36.

[17] Ulrich et Knief, Wolfgang Forstmeier, Violating the normality assumption may be the lesser of two evils, Behav. Res. Methods 53 (6) (2021) 2576–2590.

[18] Dongbo Dai, Tao Xu, Xiao Wei, et al., Using machine learning and feature engineering to characterize limited material datasets of high-entropy alloys, Comput. Mater. Sci. 175 (2020) 109618.

[19] Marcus Fischer, Florian Imgrund, Christian Janiesch, et al., Strategy archetypes for digital transformation: Defining meta objectives using business process management, Inf. Manag. 57 (5) (2020) 103262.

[20] Sanae Borrohou, Rachida Fissoune, Hassan et Badir, Data cleaning survey and challenges–improving outlier detection algorithm in machine learning, J. Smart Cities Soc. 2 (3) 125–140.

[21] Krisztián Boros, Zoltán et Kmetty, Identifying missing data handling methods with text mining, Int. J. Data Sci. Anal. (2024) 1–13.

[22] Aditya Dubey, Akhtar et Rasool, Data mining based handling missing data, in: 2019 Third International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC), IEEE, 2019, pp. 483–489.

[23] Amanda N. Baraldi, Craig K. et Enders, An introduction to modern missing data analyses, J. Sch. Psychol. 48 (1) (2010) 5–37.

[24] Mia S. Tackney, Daniel Stahl, Elizabeth Williamson, et al., Missing step count data? step away from the expectation–maximization algorithm, J. Meas. Phys. Behav. 5 (4) (2022) 205–214.

[25] Y. Angeline Christobel, R. Jaya Suji, J. Jeya A. Celin, Imputation techniques in machine learning – a survey, Int. J. Recent. Innov. Trends Comput. Commun. 11 (10) (2023) 1217.

[26] Sixia Chen, Chao et Xu, Predictive mean matching imputation procedure based on machine learning models for complex survey data, J. Data Sci. 22 (3) (2024) 456–468.

[27] Yongshi Deng, Thomas et Lumley, Multiple imputation through xgboost, J. Comput. Graph. Statist. 33 (2) (2024) 352–363.

[28] Jinsung Yoon, James Jordon, Mihaela et Schaar, Gain: Missing data imputation using generative adversarial nets, in: International Conference on Machine Learning, PMLR, 2018, pp. 5689–5698.

[29] Collins Leke, Tshilidzi Marwala, Satyakama et Paul, Proposition of a theoretical model for missing data imputation using deep learning and evolutionary algorithms, 2015, arXiv preprint a.

[30] Reza Shahbazian, Sergio et Greco, Generative adversarial networks assist missing data imputation: A comprehensive survey & evaluation, IEEE Access (2023).

[31] Yige Sun, Jing Li, Yifan Xu, et al., Deep learning versus conventional methods for missing data imputation: A review and comparative study, Expert Syst. Appl. 227 (2023) 120201.

[32] Hongzhi Wang, Mohamed Jaward Bah, Mohamed et Hammad, Progress in outlier detection techniques: A survey, IEEE Access 7 (2019) 107964-108000.

[33] Ihab F. Ilyas, Xu Chu, Data Cleaning, Association for Computing Machinery, New York, NY, USA, 2019.

[34] Markus Goldstein, Seiichi et Uchida, A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data, PloS One 11 (4) (2016) e0152173.

[35] Tlamelo Emmanuel, Thabiso Maupong, Dimane Mpoeleng, et al., A survey on missing data in machine learning, J. Big Data 8 (2021) 1–37.

[36] Indrajit Bhattacharya, Lise et Getoor, Iterative record linkage for cleaning and integration, in: Proceedings of the 9th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery, 2004, pp. 11–18.

[37] Surajit Chaudhuri, Kris Ganjam, Venkatesh Ganti, et al., Robust and efficient fuzzy match for online data cleaning, in: Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data, 2003, pp. 313–324.

[38] Peter Christen, A survey of indexing techniques for scalable record linkage and deduplication, in: IEEE Transactions on Knowledge and Data Engineering, vol. 24, (9) 2011, pp. 1537–1555.

[39] Yu Huang, Fei et Chiang, Refining duplicate detection for improved data quality, 2017, In : TDDL/MDQual/Futurity@ TPDL.

[40] Pim Verschuuren, Serena Palazzo, Tom Powell, et al., Supervised machine learning techniques for data matching based on similarity metrics, 2020, arXiv preprint arXiv:2007.04001.

[41] Sashi Tarun, Ranbir Singh Batth, Sukhpreet et Kaur, A scheme for data deduplication using advance machine learning architecture in distributed systems, in: 2021 International Conference on Computing Sciences, ICCS, IEEE, 2021, pp. 53–60.

[42] M. Ravikanth, Sampath Korra, Gowtham Mamidisetti, et al., An efficient learning based approach for automatic record deduplication with benchmark datasets, Sci. Rep. 14 (1) (2024) 16254.

[43] Viji Devarajan, Revathy et Subramanian, Analyzing semantic similarity amongst textual documents to suggest near duplicates, Indones. J. Electr. Eng. Comput. Sci. 25 (3) (2022) 1703–1711.

[44] Amro Abbas, Kushal Tirumala, Dániel Simig, et al., Semdedup: Data-efficient learning at web-scale through semantic deduplication, 2023, arXiv preprint arXiv:2303.09540.

[45] Alexandre Veríssimo De Sousa Marinho, Approximate String Matching and Duplicate Detection in the Deep Learning Era, (Thèse de doctorat), MSc thesis in Computer Engineering, Tecnico, Lisbon, 2018.

[46] Michele De Bonis, Filippo Minutella, Fabrizio Falchi, et al., A graph neural network approach for evaluating correctness of groups of duplicates, in: International Conference on Theory and Practice of Digital Libraries, Springer Nature Switzerland, Cham, 2023, pp. 207–219.

[47] Evgeny Krivosheev, Katsiaryna Mirylenka, Mattia Atzeni, et al., Graph neural networks for entity matching, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 6212–6214.

[48] Dalwinder Singh, Birmohan et Singh, Investigating the impact of data normalization on classification performance, Appl. Soft Comput. 97 (2020) 105524.

[49] Haval A. Ahmed, Peshawa J. Muhammad Ali, Abdulbasit K. Faeq, et al., An investigation on disparity responds of machine learning algorithms to data normalization method, Aro- Sci. J. Koya Univ. 10 (2) (2022) 29–37.

[50] Ismail Bin Mohamad, Dauda et Usman, Research article standardization and its effects on k-means clustering algorithm, Res. J. Appl. Sci. Eng. Technol. 6 (17) (2013) 3299–3303.

[51] Swarna Priya RM, Praveen Kumar Reddy Maddikunta, M. Parimala, et al., An effective feature engineering for DNN using hybrid PCA-gwo for intrusion detection in IoMT architecture, Comput. Commun. 160 (2020) 139–149.

[52] Rizgar Zebari, Adnan Abdulazeez, Diyar Zeebaree, et al., A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction, J. Appl. Sci. Technol. Trends 1 (1) (2020) 56–70.

[53] Utkarsh Mahadeo Khaire, R. et Dhanalakshmi, Stability of feature selection algorithm: A review, J. King Saud University- Comput. Inf. Sci. 34 (4) (2022) 1060–1073.

[54] Rizgar Zebari, Adnan Abdulazeez, Diyar Zeebaree, et al., A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction, J. Appl. Sci. Technol. Trends 1 (1) (2020) 56–70.

[55] Mengmeng Li, Haofeng Wang, Lifang Yang, et al., Fast hybrid dimensionality reduction method for classification based on feature selection and grouped feature extraction, Expert Syst. Appl. 150 (2020) 113277.

[56] Michael Tetelman, Continuous learning: Engineering super features with feature algebras, 2013, arXiv preprint arXiv:1312.5398.

[57] Xiangning Chen, Qingwei Lin, Chuan Luo, et al., Neural feature search: A neural architecture for automated feature engineering, in: 2019 IEEE International Conference on Data Mining, ICDM, IEEE, 2019, pp. 71–80.

[58] Meng Wang, Zhijun Ding, Meiqin et Pan, Lbr: a new regression architecture for automated feature engineering, in: 2020 International Conference on Data Mining Workshops, ICDMW, IEEE, 2020, pp. 432–439.

[59] Hassan Eldeeb, Shota Amashukeli, Radwa et Elshawi, Bigfeat: Scalable and interpretable automated feature engineering framework, in: 2022 IEEE International Conference on Big Data (Big Data), IEEE, 2022, pp. 515–524.

[60] Wenbin Zhu, Runwen Qiu, Ying et Fu, Comparative study on the performance of categorical variable encoders in classification and regression tasks, 2024, arXiv preprint arXiv:2401.09682.

[61] Florian Pargent, Florian Pfisterer, Janek Thomas, et al., Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features, Comput. Statist. 37 (5) (2022) 2671–2692.

[62] Ivan Lopez-Arevalo, Edwin Aldana-Bobadilla, Alejandro Molina-Villegas, et al., A memory-efficient encoding method for processing mixed-type data on machine learning, Entropy 22 (12) (2020) 1391.

[63] Hajar Hakkoum, Ali IdrI, Ibtissam Abnane, et al., Does categorical encoding affect the interpretability of a multilayer perceptron for breast cancer classification?, 2023, pp. 351–358, In : DATA.

[64] Emily Evenden, Robert Gilmore et Pontius Jr, Encoding a categorical independent variable for input to TerrSet's multi-layer perceptron, ISPRS Int. J. Geo-Inf. 10 (10) (2021) 686.

[65] Mohamed Hosni, Encoding techniques for handling categorical data in machine learning-based software development effort estimation, 2023, pp. 460–467, In : KDIR.

[66] K. Nanthini, D. Sivabalaselvamani, K. Chitra, et al., A survey on data augmentation techniques, in: 2023 7th International Conference on Computing Methodologies and Communication, ICCMC, IEEE, 2023, pp. 913–920.

[67] Markus Bayer, Marc-André Kaufhold, Christian et Reuter, A survey on data augmentation for text classification, ACM Comput. Surv. 55 (7) (2022) 1–39.

[68] Ioannis D. Apostolopoulos, Investigating the synthetic minority class oversampling technique (SMOTE) on an imbalanced cardiovascular disease (CVD) dataset, 2020, arXiv preprint arXiv:2004.04101.

[69] Mohammed Temraz, Mark T. et Keane, Solving the class imbalance problem using a counterfactual method for data augmentation, Mach. Learn. Appl. 9 (2022) 100375.

[70] Junpeng Fang, Caizhi Tang, Qing Cui, et al., Semi-supervised learning with data augmentation for tabular data, in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management, 2022, pp. 3928–3932.

[71] Umar Khan, Sohaib Zahid, Muhammad Asad Ali, et al., TabAug: data driven augmentation for enhanced table structure recognition, in: Document Analysis and Recognition–ICDAR 2021: 16th International Conference, Lausanne, Switzerland, (2021) 5–10, Proceedings, Part II 16, Springer International Publishing, 2021, pp. 585–601.

[72] Soma Onishi, Shoya et Meguro, Rethinking data augmentation for tabular data in deep learning, 2023, arXiv preprint arXiv:2305.10308.

[73] Pedro Machado, Bruno Fernandes, Paulo et Novais, Benchmarking data augmentation techniques for tabular data, in: International Conference on Intelligent Data Engineering and Automated Learning, Springer International Publishing, Cham, 2022, pp. 104–112.

[74] Salvador Garcia, Julian Luengo, José Antonio Sáez, et al., A survey of discretization techniques: Taxonomy and empirical analysis in supervised learning, IEEE Trans. Knowl. Data Eng. 25 (4) (2012) 734–750.

[75] Zeynel Cebeci, Figen et Yildiz, Unsupervised discretization of continuous variables in a chicken egg quality traits dataset, Turk. J. Agriculture- Food Sci. Technol. 5 (4) (2017) 315–320.

[76] Fabrice Muhlenbach, Ricco et Rakotomalala, Discretization for continuous attributes, in: Encyclopedia of Data Warehousing and Mining, IGI Global, 2005, pp. 397–402.

[77] B. Hemada, K.S.V. et Lakshmi, A study on discretization techniques, Int. J. Eng. Res. Technol. 2 (8) (2013) 1887–1892.

[78] Alfredo Cuzzocrea, Aggregation and multidimensional analysis of big data for large-scale scientific applications: models, issues, analytics, and beyond, in: Proceedings of the 27th International Conference on Scientific and Statistical Database Management, 2015, pp. 1–6.

[79] Navpreet Kaur, Ranjit Kaur, Reecha et Sharma, A systematic review of data aggregation using machine learning techniques, in: 2022 3rd International Conference on Computing, Analytics and Networks, ICAN, IEEE, 2022, pp. 1–6.

[80] Lei Chen, Raghu Ramakrishnan, Paul Barford, et al., Composite subset measures, 2006, pp. 403–414, In : VLDB.

[81] Devin Petersohn, Stephen Macke, others Xin, Towards scalable dataframe systems, 2020, arXiv preprint arXiv:2001.00888.

[82] Prasad M. Deshpande, Karthikeyan et Ramasamy, Data warehousing, multidimensional data models and OLAP, in: Encyclopedia of Database Technologies and Applications, IGI Global, 2005, pp. 134–139.

[83] Jigna Ashish Patel, Priyanka et Sharma, Online analytical processing for business intelligence in big data, Big Data 8 (6) (2020) 501–518.

[84] Jonas Traub, Philipp Marian Grulich, Alejandro Rodriguez Cuellar, et al., Scotty: General and efficient open-source window aggregation for stream processing systems, ACM Trans. Database Syst. 46 (1) (2021) 1–46.

[85] Sören Henning, Wilhelm et Hasselbring, Scalable and reliable multi-dimensional aggregation of sensor data streams, in: 2019 IEEE International Conference on Big Data (Big Data), IEEE, 2019, pp. 3512–3517.

[86] Asterios Katsifodimos, Sebastian et Schelter, Apache flink: Stream analytics at scale, in: 2016 IEEE International Conference on Cloud Engineering Workshop (IC2EW), IEEE, 2016, p. 193.

[87] Huy Vo, Claudio et Silva, Scaling up through parallel and distributed computing, in: Big Data and Social Science. Chapman and Hall/CRC, 2020, pp. 101–117.

[88] Reena Bharathi, Nitin N. Keswani, Siddesh D. et Shinde, An approach to mining massive data, in: Proceedings of the MPGI National Multi Conference, International Journal of Computer Applications, 2012, pp. 32–36.

[89] Yuan Yu, Pradeep Kumar Gunda, Michael et Isard, Distributed aggregation for data-parallel computing: interfaces and implementations, in: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, 2009, pp. 247–260.

[90] Wuman Luo, Haoyu Tan, Huajian Mao, et al., Efficient similarity joins on massive high-dimensional datasets using mapreduce, in: 2012 IEEE 13th International Conference on Mobile Data Management, IEEE, 2012, pp. 1–10.

[91] HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm P. Flajolet, Éric Fusy, Olivier Gandouet, Frédéric Meunier.

[92] Daniel Ting, TING daniel approximate distinct counts for billions of datasets, in: Proceedings of the 2019 International Conference on Management of Data, 2019, pp. 69–86.

[93] Sai Wu, Beng Chin Ooi, Kian-Lee et Tan, Online aggregation, in: Advanced Query Processing: Volume 1: Issues and Trends, 2013, pp. 187–210.

[94] Hector Menendez, Clustering: finding patterns in the darkness, Open J. Mach. Learn. 1 (1) (2021) 1–28.

[95] M. Namratha, T.R. et Prajwala, A comprehensive overview of clustering algorithms in pattern recognition, IOSR J. Comput. Eng. 4 (6) (2012) 23–30.

[96] Manish Suyala, Sanjay et Sharmab, A review on analysis of K-means clustering machine learning algorithm based on unsupervised learning, 2024.

[97] K. Shahina, V. et Vaidehi, Clustering and data aggregation in wireless sensor networks using machine learning algorithms, in: 2018 International Conference on Recent Trends in Advance Computing, ICRTAC, IEEE, 2018, pp. 109–115.

[98] Ulle Endriss, Umberto et Grandi, Graph aggregation, in: Companion Proceedings of the the Web Conference 2018, 2018, pp. 447–450.

[99] Yuanyuan Tian, Richard A. Hankins, Jignesh M. et Patel, Efficient aggregation for graph summarization, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 567–580.

[100] Amine Louati, Marie-Aude Aufaure, others Lechevallier, Graph aggregation: Application to social networks, 2011, pp. 157–177, In : HDSDA.

[101] Alexandros Labrinidis, Hosagrahar V. et Jagadish, Challenges and opportunities with big data, in: Proceedings of the VLDB Endowment, vol. 5, (12) 2012, pp. 2032–2033.

[102] Barbara Calabrese, Data integration and transformation, Encycl. Bioinform. Comput. Biology (2019).

[103] Mehri Mahdikhani, Bita et Yazdani, Transformational leadership and service quality in e-commerce businesses: The role of trust and team performance, Int. J. Law Manag. 62 (1) (2020) 23–46.

[104] Maurizio Massaro, Digital transformation in the healthcare sector through blockchain technology, Insights from Acad. Res. Bus. Dev. Technovation 120 (2023) 102386.

[105] Samuel Aderemi, David Olanrewaju Olutimehin, Uchenna Innocent Nnaomah, et al., Big data analytics in the financial services industry: Trends, challenges, and future prospects: A review, Int. J. Sci. Technol. Res. Arch. 6 (1) (2024) 147–166.

[106] Anuraj Soni, Reena et Duggal, Reducing risk in KYC (know your customer) for large Indian banks using big data analytics, Int. J. Comput. Appl. 97 (9) (2014).

[107] Nitin Gupta, Shashank Mujumdar, Hima Patel, et al., Data quality for machine learning tasks, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021, pp. 4040–4041.

[108] Venkat Gudivada, Amy Apon, Junhua et Ding, Data quality considerations for big data and machine learning: Going beyond data cleaning and transformations, Int. J. Adv. Softw. 10 (1) (2017) 1–20.

[109] M.R. Sundarakumar, G. Mahadevan, R. Natchadalingam, et al., A comprehensive study and review of tuning the performance on database scalability in big data analytics, J. Intell. Fuzzy Systems 44 (3) (2023) 5231–5255.

[110] Mikko Puonti, Timo Raitalaakso, Timo Aho, et al., Automating transformations in data vault data warehouse loads, in: Information Modelling and Knowledge Bases XXVIII, IOS Press, 2017, pp. 215–230.

[111] Mishaal M. Almutairi, Mohammad Yamin, George et Halikias, An analysis of data integration challenges from heterogeneous databases, in: 2021 8th International Conference on Computing for Sustainable Global Development (INDIACom), IEEE, 2021, pp. 352–356.

[112] Alexander Egyed, Klaus Zeman, Peter Hehenberger, et al., Maintaining consistency across engineering artifacts, Computer 51 (2) (2018) 28–35.

[113] Alper Bozkurt, Furkan Ekici, Hatice et Yetiskul, Utilizing flink and kafka technologies for real-time data processing: A case study, Eurasia Proc. Sci. Technol. Eng. Math. 24 (2023) 177–183.

[114] Andrea Díaz, Laura Guerra, Eleazar et Díaz, Digital transformation impact in security and privacy, in: Developments and Advances in Defense and Security: Proceedings of MICRADS 2021, Springer, Singapore, 2022, pp. 61–70.

[115] Hemant Kumud, Handling unstructured data for semantic web-a natural language processing approach, Sch. J. Eng. Technol. (SJET) 2 (2A) (2014) 193–196.

[116] Chunli Xie, Jerry Gao, Chuanqi et Tao, Big data validation case study, in: 2017 IEEE Third International Conference on Big Data Computing Service and Applications (BigDataService), IEEE, 2017, pp. 281–286.

[117] Rajiv Ranjan, Prem Prakash Jayaraman, Massimo Villari, et al., A note on resource management techniques and systems for big data workflow processing, Computing 100 (2018) 1–2.

[118] Carlos Ordonez, A comparison of data science systems, in: Big Data Analytics: 8th International Conference, BDA 2020, Sonepat, India, (2020) 15–18, Proceedings 8, Springer International Publishing, 2020, pp. 3–11.

[119] Lukas Beierlieb, André Bauer, Robert Leppich, et al., Efficient data processing: Assessing the performance of different programming languages, in: Companion of the 2023 ACM/SPEC International Conference on Performance Engineering, 2023, pp. 83–87.

[120] Paul Hudak, Domain-specific languages, Handb. Program. Lang. 3 (39–60) (1997) 21.

[121] Constantin Nandra, Dorian et Gorgan, Usability evaluation of a domain-specific language for defining aggregated processing tasks, in: 2019 IEEE 15th International Conference on Intelligent Computer Communication and Processing, ICCP, IEEE, 2019, pp. 87–94.

[122] Jesús Sánchez Cuadrado, Esther Guerra, Juan et De lara, Towards the systematic construction of domain-specific transformation languages, in: Modelling Foundations and Applications: 10th European Conference, ECMFA 2014, Held As Part of STAF 2014, York, UK, (2014) 21-25. Proceedings 10, Springer International Publishing, 2014, pp. 196–212.

[123] Jendrik Johannes, Steffen Zschaler, Miguel A. Fernández, et al., Abstracting complex languages through transformation and composition, in: Model Driven Engineering Languages and Systems: 12th International Conference, MODELS 2009, Denver, CO, USA, (2009) 4-9. Proceedings 12, Springer Berlin Heidelberg, 2009, pp. 546–550.

[124] Stefan Höppner, Timo Kehrer, Matthias et Tichy, Contrasting dedicated model transformation languages versus general purpose languages: a historical perspective on ATL versus java based on complexity and size, Softw. Syst. Model. 2022, 1–33.

[125] Didier Verna, Extensible languages: Blurring the distinction between DSL and GPL, in: Formal and Practical Aspects of Domain-Specific Languages: Recent Developments, IGI Global Scientific Publishing, 2013, pp. 1–31.

[126] Lucas Czech, Pierre Barbera, Alexandros et Stamatakis, Genesis and gappa: processing, analyzing and visualizing phylogenetic (placement) data, Bioinformatics 36 (10) (2020) 3263–3265.

[127] Bruno Martins, Nuno Freire, José et Borbinha, Complex data transformations in digital libraries with spatio-temporal information, in: Digital Libraries: Universal and Ubiquitous Access To Information: 11th International Conference on Asian Digital Libraries, ICADL 2008, Bali, Indonesia, (2008) 2-5. Proceedings 11, Springer Berlin Heidelberg, 2008, pp. 174–183.

[128] Peter Buneman, Susan Davidson, Kyle Hart, et al., A data transformation system for biological data sources, in: Proceedings of 21st VLDB Conference. Morgan Kaufmann, 1995, pp. 158–169.

[129] Vandana Singh, Lila et Holt, Learning and best practices for learning in open-source software communities, Comput. Educ. 63 (2013) 98–108.

[130] Tomaž Kosar, Sudev Bohra, Marjan et Mernik, Domain-specific languages: A systematic mapping study, Inf. Softw. Technol. 71 (2016) 77–91.